

实验零 FPGA 硬件开发初步

2026 年秋季学期

The Hitchhiker's Guide to the Galaxy itself has outsold the Encyclopedia Galactica because it is slightly cheaper, and because it has the words 'DON'T PANIC' in large, friendly letters on the cover.

– “The Hitchhiker's Guide to the Galaxy”, Douglas Adams

在本次实验中，我们以 Vivado 2022.1 为例，介绍如何利用 FPGA 开发平台 Nexys A7-100T 进行数字逻辑设计。课后可以参考相关资料，在自己的电脑上安装 Vivado 开发软件套装，并在开发板上实际进行测试。

0.1 FPGA 设计流程

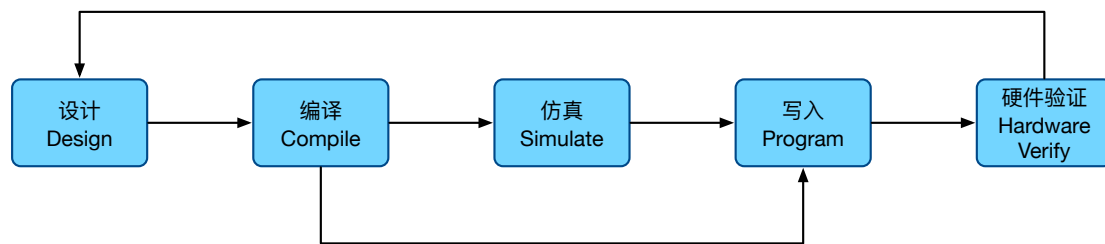


图 0-1: FPGA 设计流程

FPGA 开发流程如图 0-1 所示。其基本流程是从设计、编译、仿真、写入器件到硬件验证的一个过程。实现过程中如果出现问题，可能会反复修改设计或者代码。

FPGA 的编译一般包含多个步骤，具体的命名在不同教材和工具中略有不同。其中基本的编译完成语法分析和逻辑实现，在完成基本编译后，可以进行前端的功能仿真，对设计的逻辑功能进行验证。在仿真没有问题后，可以进行综合和布局布线。这时基本的逻辑功能将映射到实际物理器件上，生成网表和 FPGA 二进制烧写文件。随后可以进行后端仿真，针对物理器件的实际延时和物理性能，分析最终器件是否能满足设计要求。最后，可以将 FPGA 二进制文件写入 FPGA，在实际物理硬件上进行验证。

0.2 实验环境介绍

0.2.1 实验软硬件环境

本学期的实验将使用以下的硬件和软件环境：

硬件： Digilent Nexys A7-100T 开发板 (原 Nexys 4 DDR)，核心芯片为 Xilinx Artix-7 FPGA 芯片。该开发板的相关手册可以在课程网站上下载。

软件： Vivado 2022.1。可以自行在 Xilinx（AMD）网站上下载安装。



请注意 Vivado 软件体积较大，下载约需 60G 左右空间，请预留 200G 左右空间进行安装。

0.2.2 开发板简介

Digilent Nexys A7-100T (原名:Nexys 4 DDR) 采用了 Xilinx Artix-7 FPGA (XC7A100T-1CSG324C) 芯片，是一款简单易用的数字电路开发平台，可以支持在教学环境中设计及实现各类系统应用。具有大规模、高容量的 FPGA，包括 USB、以太网及 VGA 在内的外设接口，能够满足从入门级组合逻辑电路到强大的嵌入式系统的设计实现需求。开发板上还集成了加速度、温度传感器，MEMs 数字麦克风，扬声器以及大量的 I/O 设备，特别是包含了 128MB 的 DDR2 内存，让 Digilent A7-100T 可以在不需要增添额外组件的条件下用于各种各样的设计。

Nexys A7-100T 的核心 FPGA 为 Artix-7 系列，包含 63,400 个 Look-Up Tables (LUT) 资源，126,800 个触发器及 4,860Kb 的 SRAM 资源。同时，芯片内部还有 240 个 DSP Slices 及 6 个时钟管理模块，内部时钟可以达到 450MHz。

Nexys A7-100T 的开发板布局及板载接口如图 0-2 所示：

Nexys A7-100T 的主要板载接口包括：

1. 拨动开关 (Switch)：16 个拨动开关 (SW0-SW15)，当开关拨到上档时，表示 FPGA 输入为高电平；当开关拨到下档时，表示 FPGA 输入为低电平。
2. LED 指示灯 (LED)：16 个 LED 指示灯 (LD0-LD15)，当 FPGA 引脚输出为高电平时，相应的 LED 指示灯点亮，否则 LED 指示灯熄灭。
3. 按钮 (Button)：5 个按钮 (BTNU/D/L/R/C)，表示 FPGA 相应的输入引脚为高电平。在实际应用和实验设计中，可以利用按钮来定义复位输入，或者可以用于对应上下左右中五个方向按键 (你懂的)。

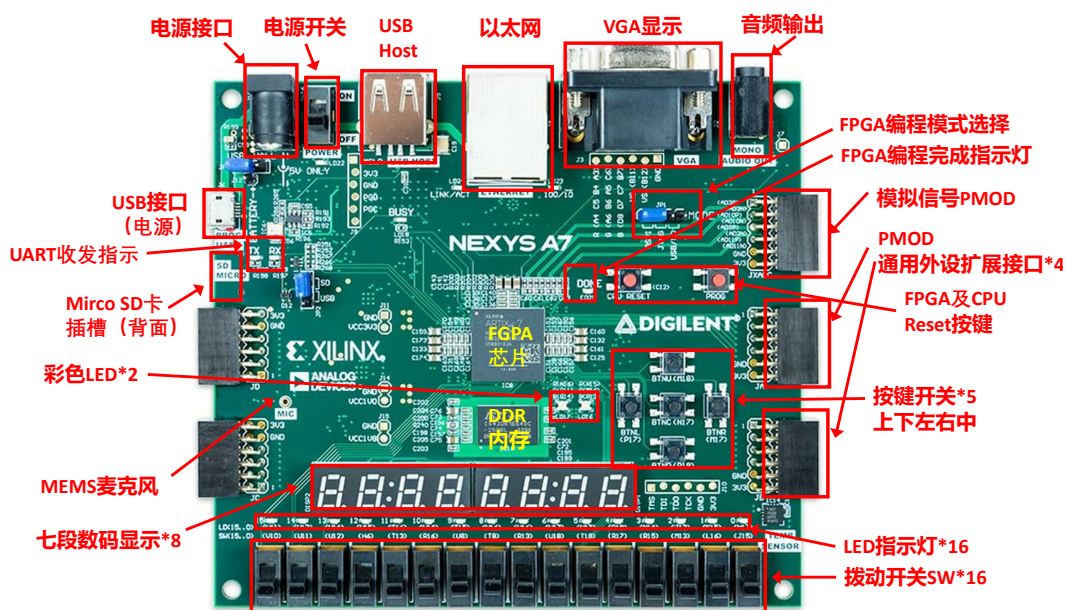


图 0-2: Nexys A7-100T 开发板布局

4. 七段数码管（Seven Segment Display）：8 个七段数码显示，采用共阳极方式驱动，显示数字及小数点。具体使用参见后面的实验。
5. VGA 输出：用于接 VGA 显示器，具体使用参见后面的实验。
6. USB HID Host 接口：可以接鼠标、键盘和 U 盘等等，具体使用参见后面的实验。

以下接口为后续扩展实验使用

7. 单声道 PWM 音频输出
8. 单声道 PDM 麦克风
9. 三色 LED：2 个彩色 LED。
10. micro SD 卡槽：外接存储。
11. 10/100M 以太网接口
12. 3 轴加速度计
13. 温度传感器
14. 串行 Flash 存储器
15. 模拟信号 PMOD 接口
16. 通用 PMOD 接口：4 个用于扩展外设的通用 PMOD 接口。

上述硬件接口均与 FPGA 的给定引脚相连，通过设置引脚的电平可以对外

设进行控制，读取给定引脚可以获取外设的输入。例如，观察板载拨动开关 SW0 上小字标注引脚号 J15，因此，FPGA 的引脚 J15 对应开关 SW0。当 SW0 向上拨的时候，对应输入信号变为高电平。同样，LED 指示灯 LD0 对应引脚 H17，则电路中对 H17 的信号置高电平时，对应的 LED 灯会亮。

 在实际应用中，如果把输入和输出引脚定义错误，则可能导致短路现象，进而损坏芯片和器件。因此在使用开发板输入输出端口时，必须严格按照 FPGA 芯片引脚定义来分配使用。Nexys A7-100T 的引脚分配可以查阅我们提供的开发板手册。

0.2.3 Vivado 开发工具简介

Vivado 设计套件是 FPGA 厂商 Xilinx 公司 2012 年首次发布的集成设计环境，包含设计输入、逻辑仿真、设计综合、布局布线、时序分析、下载配置、验证测试等一体化的 FPGA 开发工具。Vivado 不仅包含传统上的寄存器传输级 (RTL) 到比特流的 FPGA 的设计流程，而且提供了基于知识产权 IP 核的系统级设计。

Vivado 设计流程分为 Project Mode 和 Non-project Mode 两种模式，一般简单设计中，常用 Project Mode 模式。设计的过程包括：创建 Vivado 工程，创建模块，创建用户约束文件，插入创建的模块，声明创建的约束文件，进行功能仿真，对创建的模块进行综合，执行设计，生成编程 bit 文件，最后将文件下载到开发板上验证设计的正确性等步骤。Vivado IDE 的主界面如图 0-3 所示。主要包括如下部件：

1. 菜单栏，提供了对 Vivado IDE 菜单命令的访问。
2. 主要工具栏：一键访问最常用的命令。
3. 流程导航窗口，提供项目设计的主要处理流程的命令和工具的入口。
4. 数据窗口区，显示与设计源和数据相关的信息，如 Source 窗口、Netlist 窗口和属性窗口等。
5. 菜单命令快速搜索字段：根据输入字母，快速显示菜单命令列表。
6. 工作区，显示带有图形界面的窗口，如文本编辑器、原理图窗口、设备窗口、封装窗口等。
7. 项目状态栏，当前设计流程的状态。
8. 布局选择器，提供预定义的窗口布局，促进各种任务的设计处理。

9. 状态栏: 鼠标悬停时, 显示菜单命令或工具栏按钮的描述。
10. 结果窗口区, 显示运行命令的状态和结果。包括生成消息、日志、报表和 tcl 控制台等。

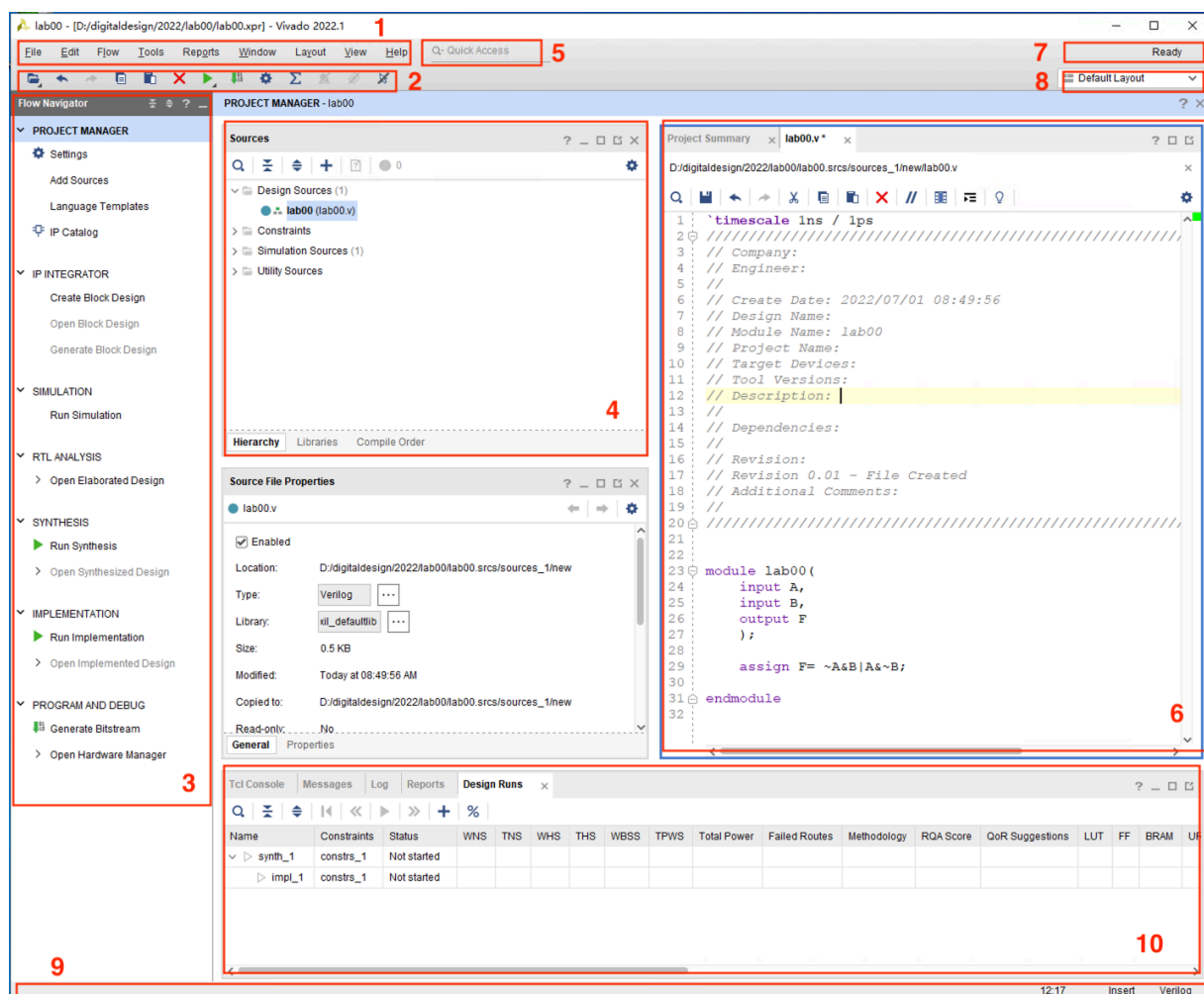


图 0-3: Vivado 主界面介绍

Vivado 主界面的左侧是流程导航窗口, 给出了工程项目的主要处理流程。

1. PROJECT MANAGER (工程项目管理器)

- Settings 项目配置: 设置项目、工具、综合、实现、IP 等相关选项。
- Add Source 添加源文件: 在工程项目添加或创建源文件, 包括设计文件、仿真测试文件和约束文件等。
- Language Templates 语言模板: 显示语言模板窗口, 如 Verilog、VHDL、System Verilog、XDC、debug 等。

- IP Catalog IP 目录: 浏览、自定义和生成 IP 核。

2. IP INTERGRATOR (IP 集成器)

- Create Block Design 创建块设计: 创建并添加一个 IP 子系统。
- Open Block Design 打开块设计: 从当前工程中打开一个 IP 子系统。
- Generate Block Design 生成块设计: 生成输出需要的仿真综合和实现的 IP 子系统。

3. SIMULATION (仿真)

- Run Simulation 运行仿真: 启动仿真命令

4. RTL ANALYSIS (RTL 分析)

- Open Elaborated Design 打开细化后的设计: 分析和约束解析后的网表文件。

5. SYTHESIS (综合)

- Run Synthesis 运行综合: 启动综合命令
- Open Synthesized Design 打开综合后的设计: 分析和约束综合后的网表文件。

6. IMPLEMENTATION (实现)

- Run Implementation 运行实现: 启动实现命令
- Open Implemented Design 打开实现后的设计: 分析和约束一个实现的设计。

7. PROGRAM AND DEBUG (编程和调试)

- Generate Bitstream 生成比特流: 实现后生成编程文件。
- Open Hardware Manager 打开硬件管理: 打开硬件管理和调试程序。

0.3 设计目标

在本实验中，我们将一步一步设计一个简单的 **FPGA** 应用：双控开关。该应用的目标是让两个拨动开关能够同时控制一盏灯。例如，在卧室中可以通过门口或床头的开关来开关卧室的灯，这样开关灯时就不用跑来跑去了。

假设两个开关分别用 **A** 和 **B** 来表示，灯用 **F** 来表示。双控开关可用一个简

单的异或操作：

$$F = A \oplus B = \bar{A}B + A\bar{B} \quad (0-1)$$

来实现。下面我们就一步一步地来在 Nexys A7-100T 平台上实现这个功能。

0.4 建立工程

双击桌面上的 Vivado 图标 (🚀) 打开 Vivado，出现如图 0-4 界面。

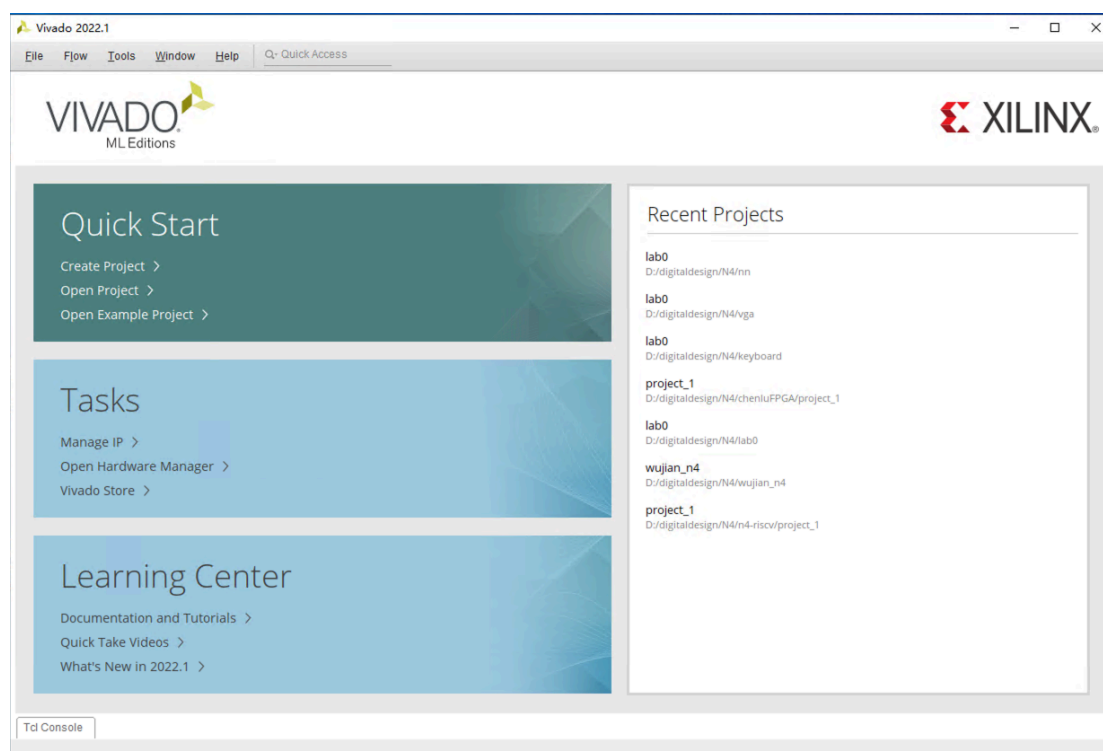


图 0-4: Vivado 2022 主界面

在 Vivado 的工作界面中点击 **Quick Start→Create Project** 或者点击 **File→Project→New**，弹出新建工程介绍页面（图 0-5）。

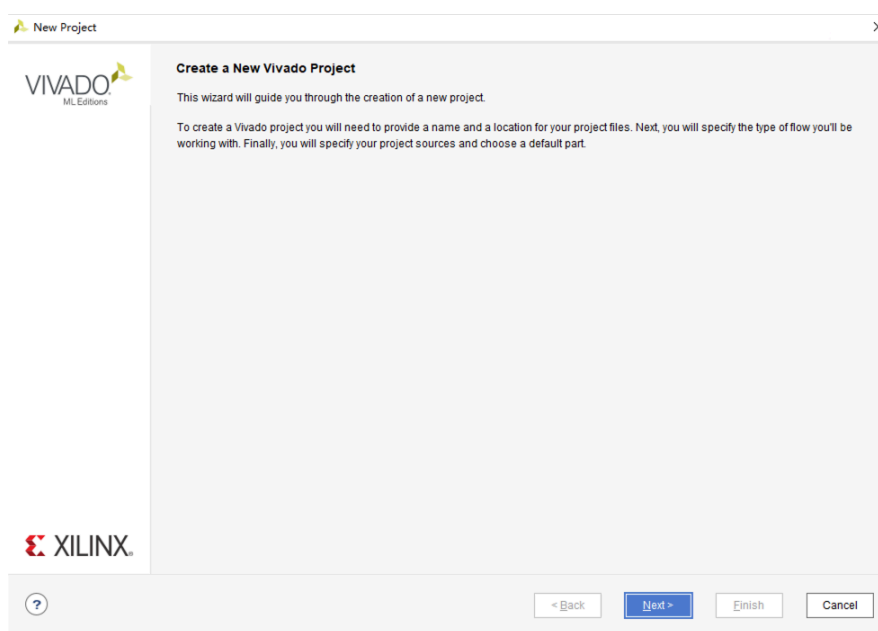


图 0-5: 新建工程界面

点击 **Next**。在弹出的界面中填写以下信息：

- 这个工程的工作路径。请你自己新建一个路径专门用于保存本学期的所有工程，比如：D:\digitaldesign\2022。
- 为本次工程取一个名字，比如：lab00。
- 默认勾选创建工程对应的子目录。

 请注意文件名、工程名和路径中都不能含有空格，并建议不要包含中文字符。工程名和文件名请用字母开头，是字母、数字和下划线的组合。

如图 0-6 所示，点击 **Next**，进入下一步。

在选择工程类型时选择建立 RTL Project，如图 0-7 所示。勾选 “Do not specify sources at this time”，勾选该选项是为了跳过在新建工程的过程中添加设计源文件的过程。如果不勾选 “Do not specify sources at this time”，在我们新建工程的时候，默认的可以添加源文件，并且可以设置编程语言和仿真测试文件语言，将其均设置为 Verilog。点击 **Next**。没有可添加的 IP，所以不添加 IP，点击 **Next**。也没有可添加的约束文件，继续点击 **Next**，进行下一步。

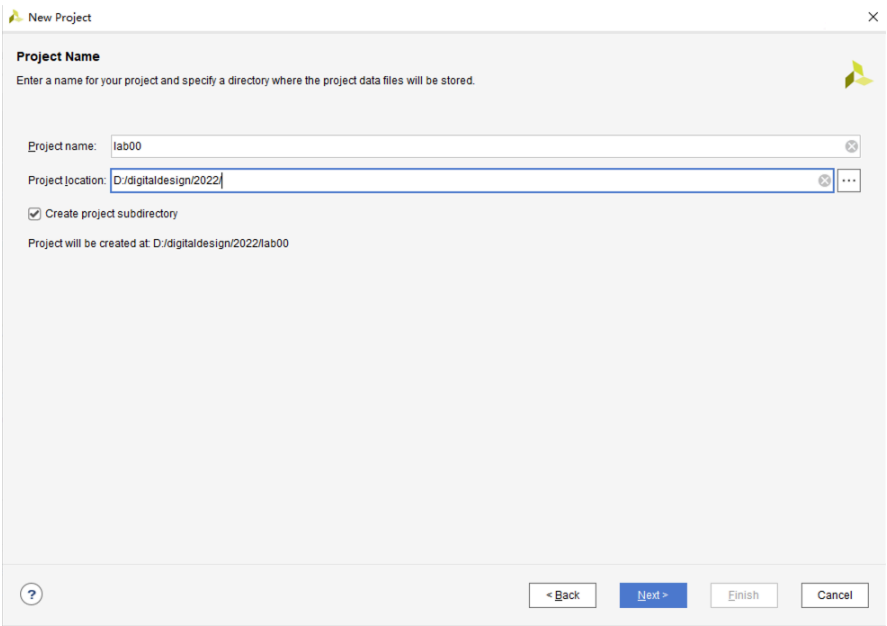


图 0-6: 输入工程名

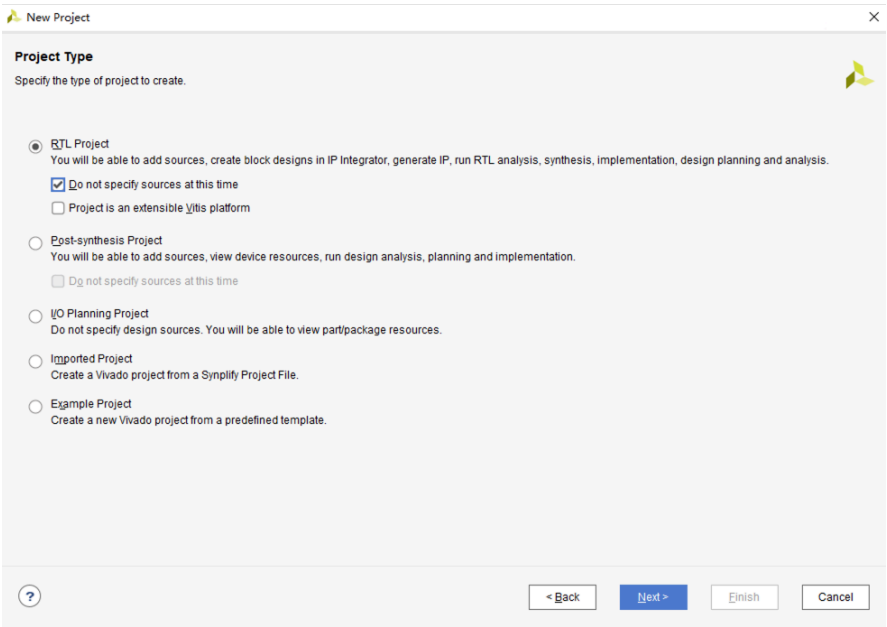


图 0-7: 项目类型选择界面

进入选择器件窗口. 此时可以有两种选择器件的方式，第一种是通过 **FPGA 芯片型号 Parts** 来选择，如图 0-8 搜索开发板的 **FPGA 型号xc7a100tcsg324-1**选择。

第二种是使用开发板选择，首先需要将课程提供的 **vivado-boards-master.zip**

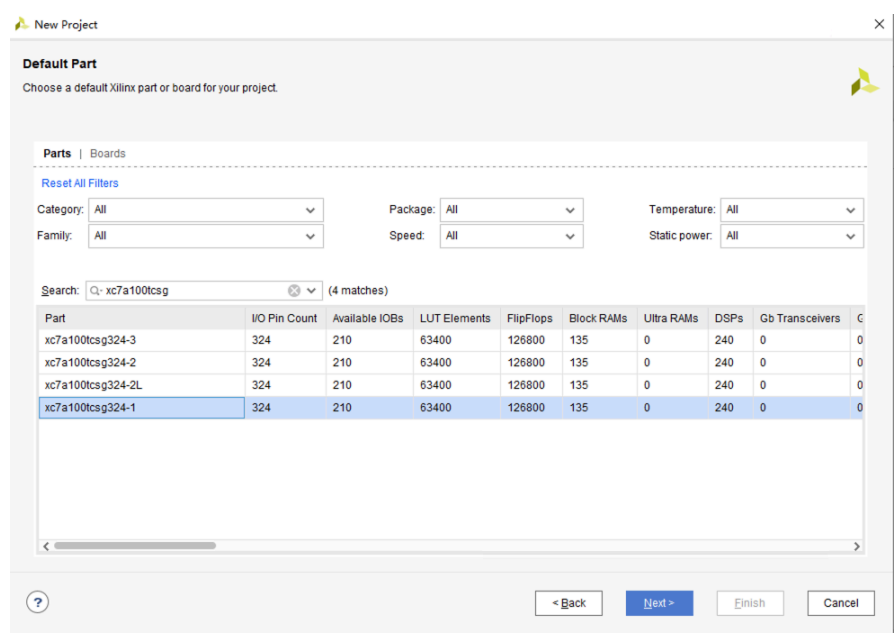


图 0-8: 选择器件，使用 Parts 方式

中的开发板信息文件 (在压缩包的 new/board_files 目录下)，拷贝到 vivado 安装目录的 data/board/board_files/中去。随后如图 0-9 就可以看到 Nexys A7-100T 对应的开发板，进行选择即可。

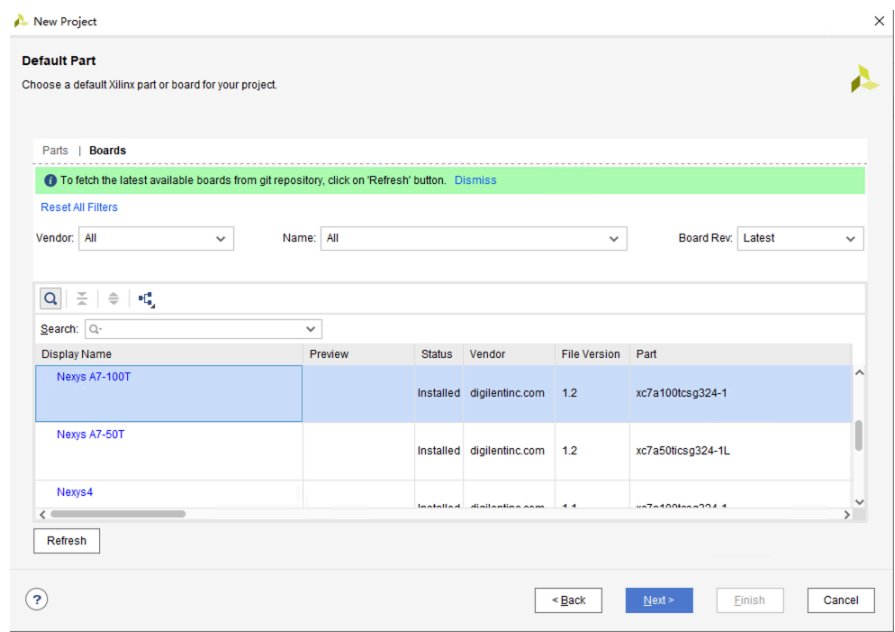


图 0-9: 选择器件，使用 Board 方式

点击[Next]，进入新建工程总结界面，确认相关信息与设计所用的 FPGA 器件信息是否一致，一致请点击[Finish]完成新建工程，否则退回之前的步骤进行修改。随后 Vivado 会进入图 0-10 的工程主界面。

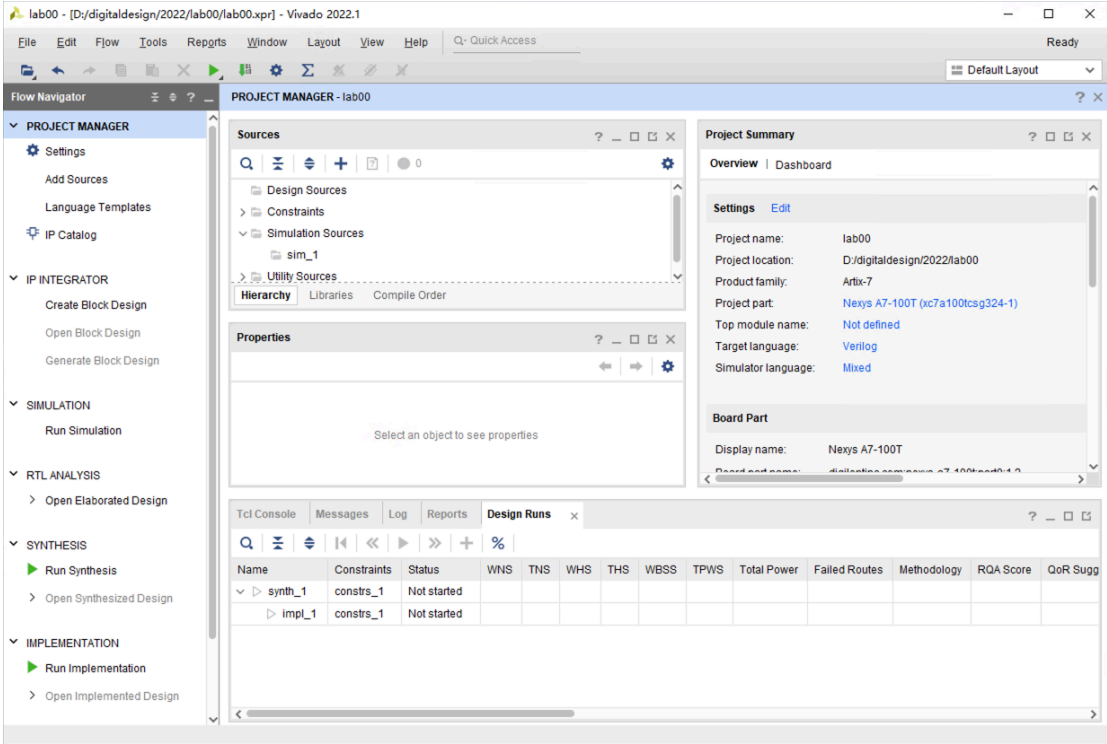


图 0-10: 工程主界面

0.5 设计并编译

在建立完一个空工程后，我们需要添加设计文件实现具体功能。首先要完成工程的顶层模块的设计，顶层模块的输入输出将直接对应 FPGA 的引脚。

0.5.1 添加设计文件

在界面左侧 Flow Navigator 中选择 **Project Manager → Add Sources** 或者界面中间 Sources 窗口里点击 + 添加设计文件，在弹出的对话框（图 0-11）中选择 **Add or create design sources**，点击 **Next**。

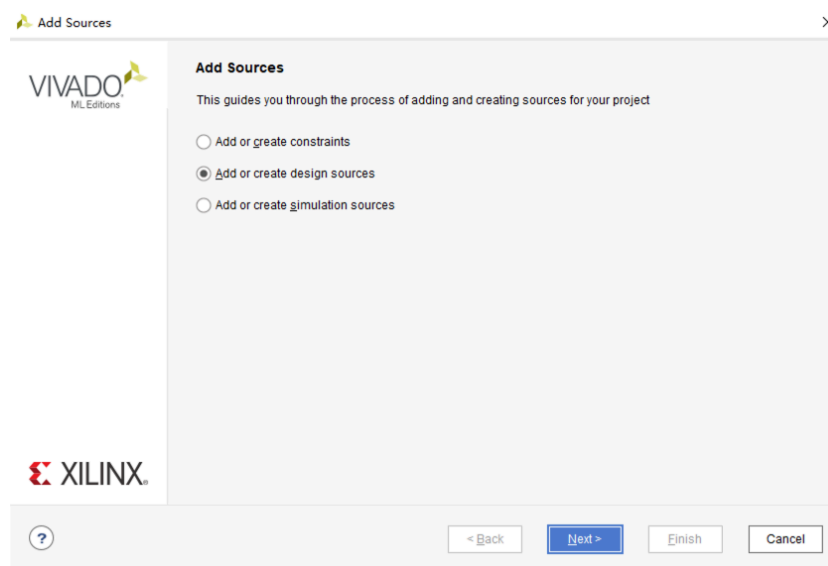


图 0-11: 添加设计文件

在图 0-12 中的对话框里可以通过点击 + 号来添加已经有的 Verilog 对应的 .v 文件。我们这里选择直接新建一个 .v 文件，点击 **Create File**。

在弹出的对话框图 0-13 中选择 Verilog 语言，并输入文件名 lab00，顶层文件名可以与工程名一样也可以不一样。注意 Vivado 的文件名是区分大小写的。

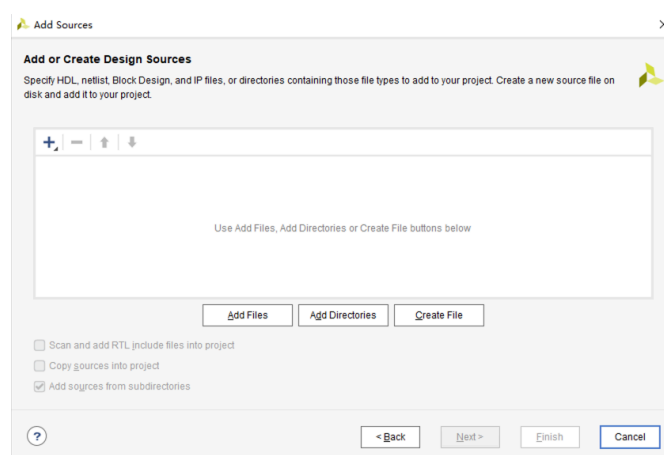


图 0-12: 选择新建设计文件

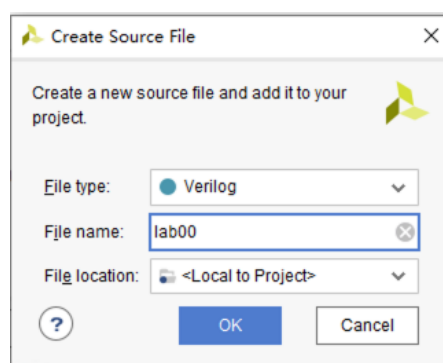


图 0-13: 输入新建文件名

新建源文件名确认后，点击`Finish`按钮，在弹出的 **Define Module** 对话框中的 **I/O Port Definition**，输入此次设计的模块中所需要的端口，请注意设置输入/输出 (input/output) 选项，如图 0-14 所示。如果端口为总线型，勾选 **Bus** 选项，并通过 **MSB** 和 **LSB** 确定总线宽度。完成后点击`OK`。这一步也可以不用设置端口，以后在 Verilog 文件中直接声明也可以。如果这一步设置错了，也可以在 Verilog 文件中修改。

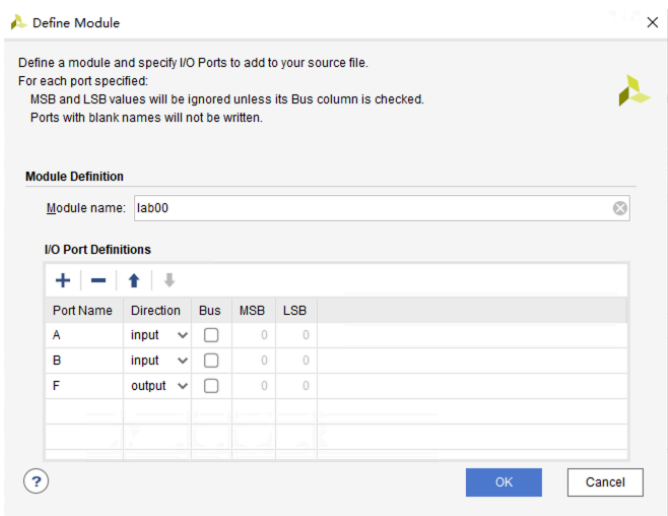


图 0-14: 输入端口定义

0.5.2 设计

如图0-15，在 Vivado 的工作区找到 Design Source 中的 lab00.v，如图输入表 0-1中的顶层设计文件，请参考教科书上第五章的 Verilog 语法理解设计文件的含义。

表 0-1: 双控开关设计文件

```
1 module lab00(  
2     input A,  
3     input B,  
4     output F  
5 );  
6  
7     assign F= ~A&B|A&~B;  
8  
9 endmodule
```

输入后，保存源文件，编译器会自动初步检查语法、拼写错误，并将此文件加入到工程库中。如果编译器检查没有语法、拼写错误，则在源程序编辑窗口的右上角显示绿色小方块，如图 0-15所示。

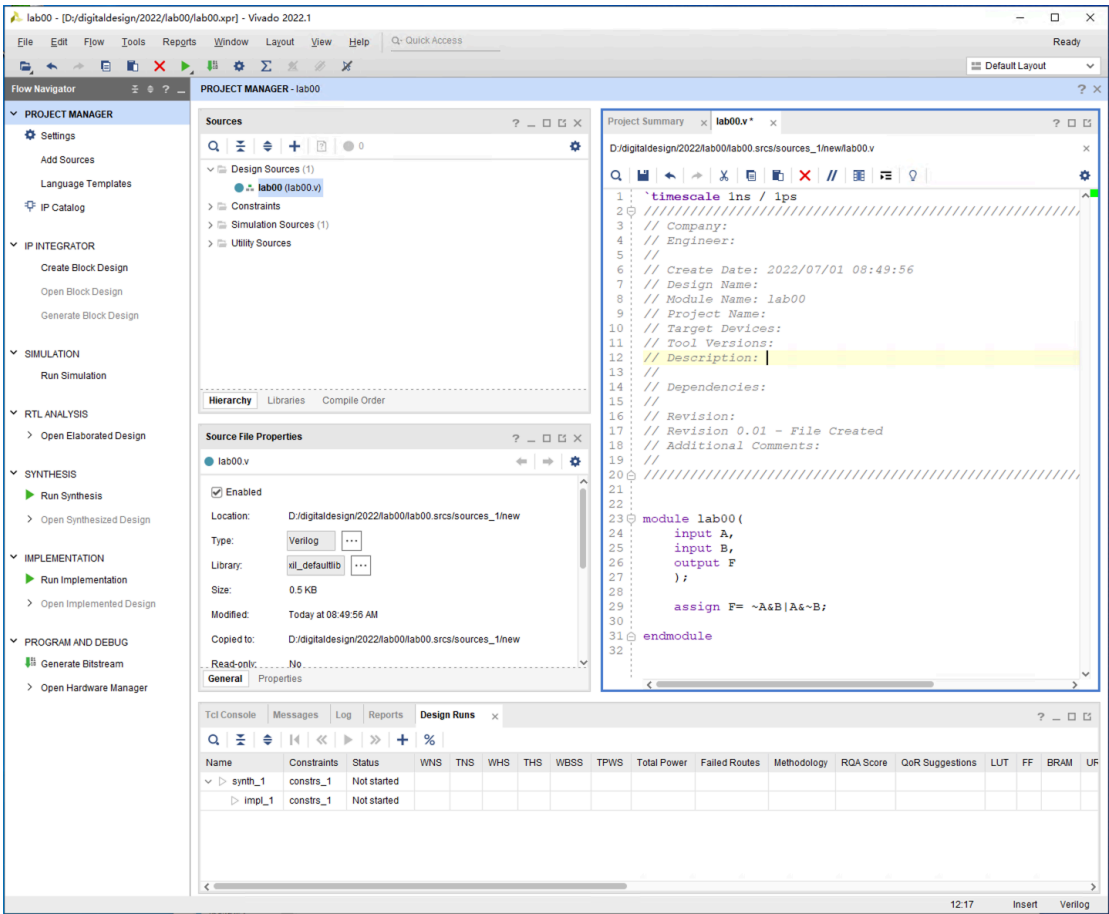


图 0-15: 输入设计文件

如果存在语法错误，则在编辑窗口的右上角显示为红色方块，并在发生语法错误的那一行右侧边框上显示为红色，源文件存盘后 **Source** 窗口的快捷工具栏上用橙色圆圈提示错误信息记录数，同时在结果窗口区的 **Message** 栏中提示语法错误信息，如图0-16所示。检查代码中是否有书写、语法等错误，消除错误后，源文件编辑恢复正常状态，进入逻辑仿真阶段。



请在编辑代码时注意变量名大小写、拼写、线宽设置等问题，并注意编译过程中出现的警告。忽视这些问题可能会造成难以预料的 **Bug**。

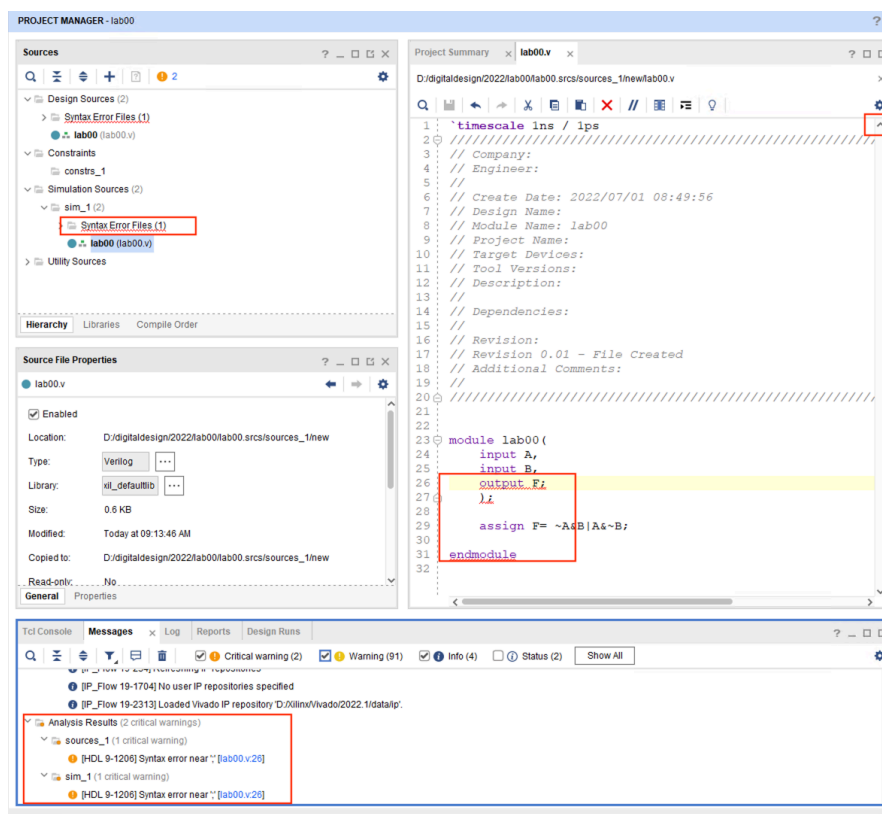


图 0-16: 错误提示

0.6 逻辑仿真

功能仿真是验证代码功能实现的基本方式。逻辑仿真是验证代码功能实现的基本方式。设计文件完成之后，要检查设计的逻辑是否符合要求，可以通过查看电路的输入输出时序图来验证，从而验证功能的正确性。

在 Vivado 中要通过添加设计文件的仿真测试程序（test bench）来编辑电路的输入时序，验证输入和输出功能是否符合预期。

添加测试程序的方法与添加设计文件类似，也是在 Project Manager 中点击 **Add Source**，如图 0-17 所示。

点击 **Next** 进入仿真测试文件创建界面，如图 0-18 所示。这里选择仿真测试文件名为 **lab00_test**。注意仿真测试文件名要与设计文件名有区别。点击 **OK** 后，在仿真测试文件创建界面中点击 **Finish**。此时会弹出 IO 定义对话框。对于仿真测试文件，我们不需要定义对外的端口，所以所有的端口空白即可，直接点击 **OK** 完成文件创建。

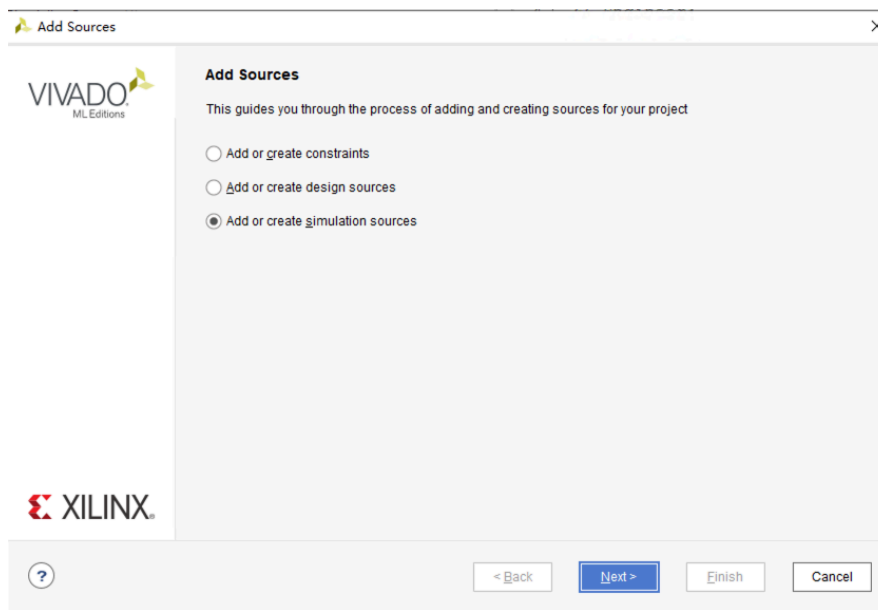


图 0-17: 选择添加测试仿真代码

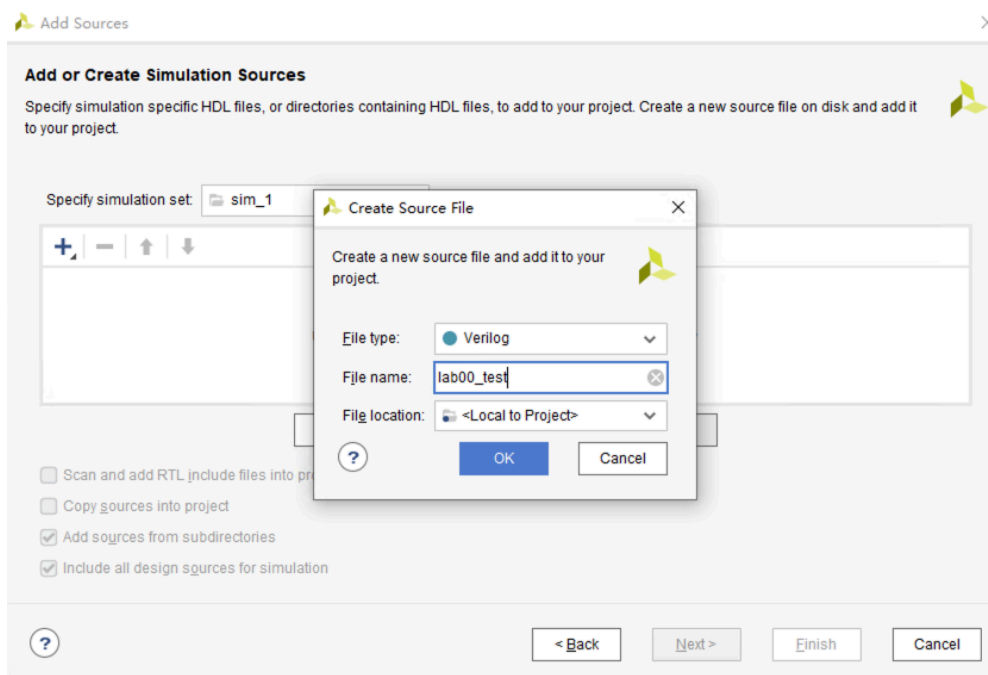


图 0-18: 添加仿真测试文件

0.6.1 编写测试文件

如图0-19所示，在 Source 窗口的 Simulation Source 的 sim_1 下，显示了新创建的测试文件名 lab00_test.v

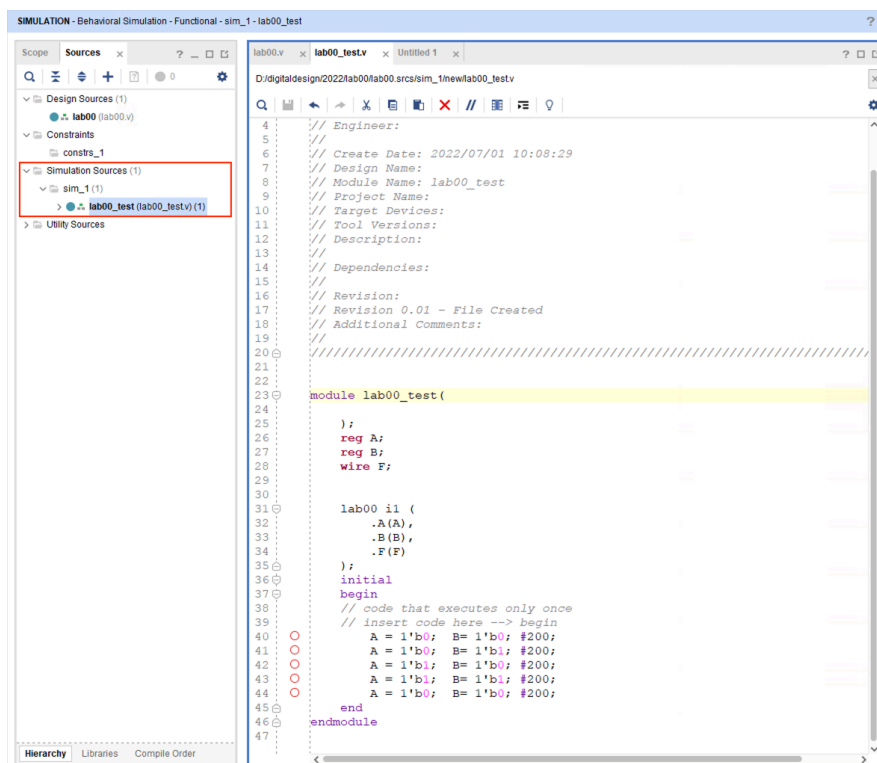


图 0-19: 编辑测试文件

打开测试文件后，根据我们自己的要求，对其进行修改保存：

我们对测试代码的修改主要是按照我们自己的要求，在测试代码中增加激励。对输入 A 和 B 按测试需求进行设置，相关语法参考教科书中测试代码的内容。

表 0-2: 双控开关测试文件修改后结果

```
1  `timescale 1 ns/ 1 ps
2
3  module lab00_test(
4      );
5
6  reg A;
7  reg B;
8  wire F;
9
10
11 lab00 i1 (
12     .A(A),
13     .B(B),
14     .F(F)
15 );
16 initial
17 begin
18     // code that executes only once
19     // insert code here --> begin
20     A = 1'b0; B= 1'b0; #200;
21     A = 1'b0; B= 1'b1; #200;
22     A = 1'b1; B= 1'b0; #200;
23     A = 1'b1; B= 1'b1; #200;
24     A = 1'b0; B= 1'b0; #200;
25 end
26
27 endmodule
```

0.6.2 测试程序解释

- 测试时间单位设置

```
`timescale 1 ns/ 1 ps
```

此语句说明时间单位为 1ns 且时间精度为 1ps。如“#200”代表延时时间 200ns。在生成测试文件的时候，编译器会在测试文件的第一行自动产生这个语句，如果自动产生了，则不需要重复输入了，但用户可以修改延时时间和时间精度。

- 待测模块实例化

```
lab00 i1 (
    .A(A),
    .B(B),
    .F(F)
);
```

这条语句是将 lab00.v 文件中的 lab00 模块实例化，并取名为 “i0”。因为需要对 lab00 模块进行仿真，因此要实例化 lab00 模块。将 lab00_test 模块中的 A, B, F 变量值分别和 lab00 中的 A, B 和 F 变量值一一对应起来。注意：在测试文件的变量声明中，输入变量, 如 A, B, 应声明为 “reg” 类型的；输出变量，如 F, 要声明为 “wire” 类型的。

● 测试激励

```
initial
begin
// code that executes only once
// insert code here --> begin
    A = 1'b0;   B= 1'b0; #200;
    A = 1'b0;   B= 1'b1; #200;
    A = 1'b1;   B= 1'b0; #200;
    A = 1'b1;   B= 1'b1; #200;
    A = 1'b0;   B= 1'b0; #200;
end
```

在 initial 语句块里，我们对输入 A 和 B 进行了设置，并在每次设置之后延迟 200ns，这样可以在改变输入激励信号时观察 F 的输出是否符合预期。

保存仿真测试文件，如果没有语法错误，这时在 Sources 栏的 Simulation Sources 栏下会出现实例 “i0”。如果没有出现 “i0”，则检查测试代码是否有逻辑错误，修改错误，保存，直至测试代码被正确解析。

0.6.3 运行仿真

点击 Flow Navigator 下的 Run Simulation，选择 **Run Behavioral Simulation**。如果仿真代码正确，会进行功能仿真，进入如图 0-20 界面。

在仿真图中，可以利用缩放工具 ，对仿真结果窗口进行调整。例如，当初始显示范围不对，看不到信号变化时，可以点击最右图标自动缩放，在合适的缩放比例下查看仿真结果是否满足设计需要。

还可以通过左侧 Scope 一栏中的目录结构定位到设计者想要查看的 module 内部寄存器，点击 Scopes 栏中的 “i0”，在 Objects 对应的信号名称上右击选择 **Add To Wave Window**，将信号加入波形图中。但本例中没有内部变量，无法添加。

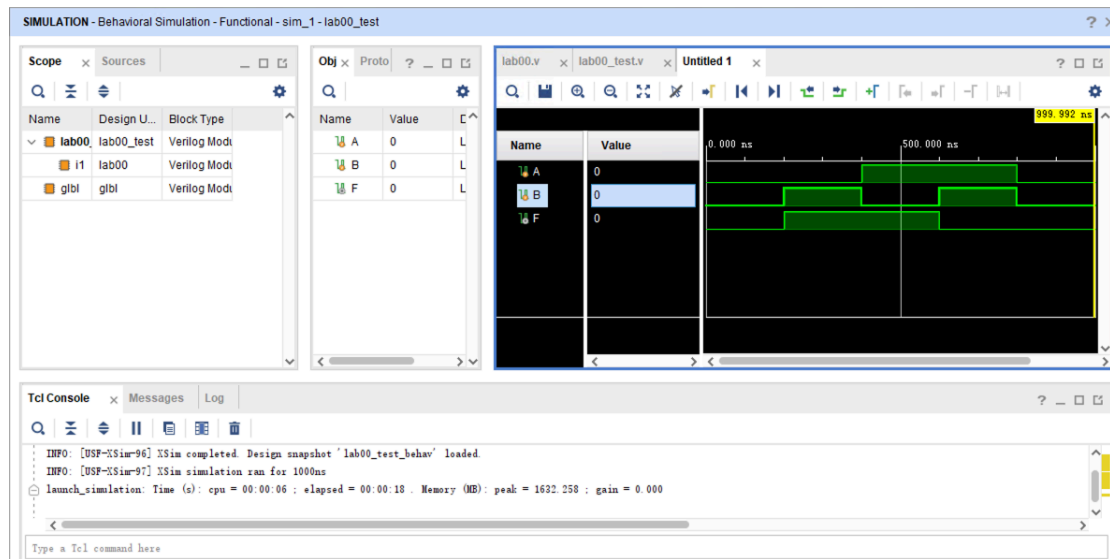
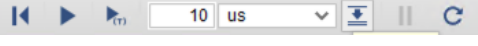


图 0-20: 运行仿真界面

我们也可通过选择工具栏中的选项来进行波形的仿真时间控制。如工具条 ，分别是复位波形 (即清空现有波形)、运行仿真、运行特定时长的仿真、仿真时长设置、仿真时长单位、单步运行、暂停、重启。

请核对波形与预设的逻辑功能是否一致。从仿真图中可以看到，当 $A=0, B=0$ 时， $F=0$ ；当 $A=0, B=1$ 时， $F=1$ ；当 $A=1, B=0$ 时， $F=1$ ；当 $A=1, B=1$ 时， $F=0$ ；可见输入输出信号满足设计要求，仿真测试功能通过。

0.7 设计综合 (Synthesis)

设计综合是把项目设计文件进行逻辑优化，并将 RTL 代码映射到 FPGA 器件的原语，生成网表文件。如果 RTL 代码中有语法问题，这一步就会报错；同时，综合后生成的报告需要仔细查看。

在流程导航窗口中的 Synthesis 下点击 **Run Synthesis** 菜单，在弹出的对话框中选择在本地运行，然后点击 **OK**，如图 0-21 所示。

综合时间一般较长，请耐心等待。在综合过程中将对设计进行检查，检查设计中的逻辑错误等，如果报错，请仔细检查“error”项，修改每一个 error 后再重新选择设计综合，直至所有错误全部修改通过。可以在弹出的对话框中选择编译报告查看，也可以直接关闭窗口。完成综合后，将弹出综合成功完成的

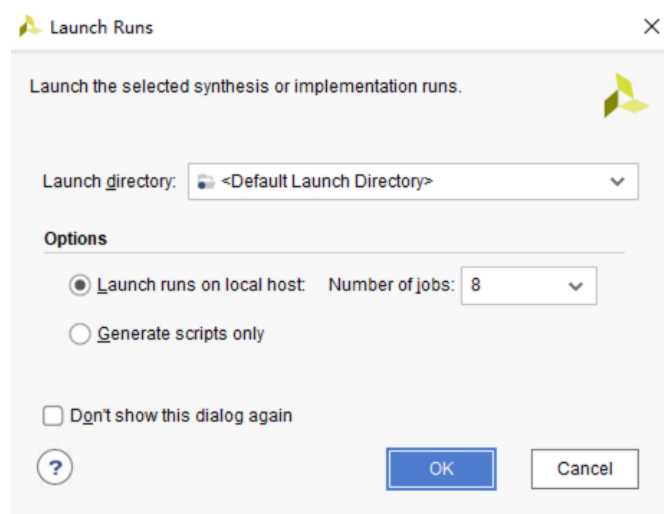


图 0-21: 启动综合与实现

对话框，如图0-22所示。有 3 个选项：

1. Run Implementation 运行实现过程
2. Open Synthesized Design 打开综合后的设计
3. View Reports 查看报告

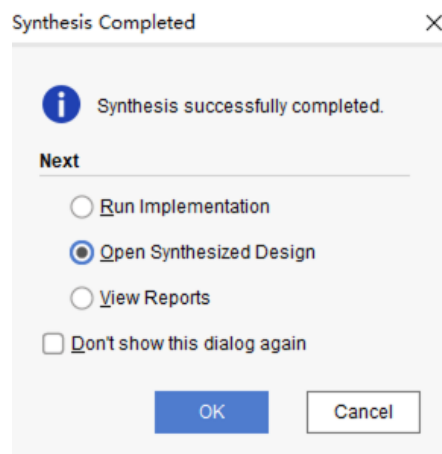


图 0-22: 综合结果报告

其中第一个选项是继续执行下一步实现过程，我们这里尚未添加约束文件，所以先选择第二个选项，查看综合后的设计。如果对设计比较熟悉，可以跳过查看步骤，直接进行实现过程。执行 **Open Synthesized Design** 选项后，在流程导航窗口中的 **Open Synthesized Design** 有如下选项：

1. Constraints Wizard: 约束向导。
2. Edit Timing Constraints: 编辑时序约束，用于启动时序约束标签。
3. Reports Timing Summary: 时序汇总报告，用于生成一个默认的时序报告。
4. Report Clock Networks: 时钟网络报告，用于创建一个时钟网络报告。
5. Report Clock Interaction: 时钟相互作用报告，用于在时钟域之间，验证路径上的约束收敛。
6. Report Methodology: 设计方法报告，用于检查设计是否符合 UltraFast 设计方法。
7. Report DRC:DRC 报告，用于对整个设计检查设计规则的报告。
8. Report Noise: 噪声报告，用于当前的引脚分配生成同步开关噪声分析报告。
9. Report Utilization: 利用率报告，用于创建资源利用率报告。
10. Report Power: 功耗报告，用于生成详细的功耗分析报告。
11. Schematic: 原理图，用于创建该设计电路的原理图。

我们选择 Schematic，打开综合后的电路原理图如图 0-23 所示： 我们看到电路

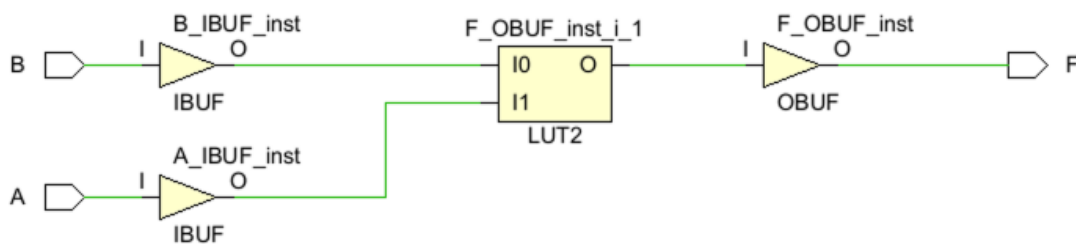


图 0-23: 综合后生成的电路原理图

实现上是使用 LUT 来实现对 A 和 B 的异或操作。

0.8 实现 (Implementation)

在综合完成对话框中，选择执行 Run Implementation 选项，Implementation 主要包含布局布线两个过程，布局主要将综合后的基本单元放到 FPGA 中合适的位置，而布线则是将这些基本单元连接起来。项目实施需要通过引脚约束和时序约束，把网表文件映射到可用 FPGA 器件资源上，在物理上将逻辑电路互连起来。

0.8.1 创建约束文件 Constraints

我们设计的电路是通过下载到 FPGA 上实现的，FPGA 自身没有输入输出设备，必须使用外部的输入输出设备。在 A7-100T 开发平台上，所有的输入输出设备都被固定地和 FPGA 的某个引脚连接在一起了。比如开发板上的拨动开关 SW0，SW1 以及发光二极管 LED0，是分别和 FPGA 的 J15，L16 和 H17 连接在一起的。设计代码中定义了输入/输出端口分别取名为 A，B 和 F。如果我们将设计中的 F 端口和 FPGA 的 H17 引脚连接起来，而 H17 在开发板上又是和指示灯 LED0 连接起来的，那么在硬件上我们就将 F 端口和 LED0 连接起来了。如果电路的 F 端输出“1”值，则开发板上的 LED0 就会亮。这样就可以使用开发板上的输入输出设备来验证我们的电路。

如何配置我们设计的电路中的输入/输出引脚，利用开发板上的输入输出硬件来验证我们设计的电路呢？这就需要对我们的电路中输入输出端口进行引脚约束，将我们电路中的输入输出端口和 FPGA 的引脚进行配置连接，从而使其连接到开发板的输入输出器件上。有两种方法可以添加引脚约束文件：一是可以直接新建 XDC 的约束文件，手动输入约束命令；二是可利用 Vivado 中 IO planning 功能。

方法一：通过约束文件来实现

与添加设计源文件步骤类似，在流程导航窗口中的 Project Manager 区域，点击 **Add Sources**，在弹出的添加文件对话框中，选择 **Add or Create Constraints**，如图 0-24，随后点击 **Next**，

约束文件的后缀是 .xdc。在约束文件中可以对设计中的信号的具体要求及引脚进行定义。为简化引脚分配的过程，课程提供了 Nexys A7 100T 开发板的缺省约束文件，文件名为 nexysa7.xdc。将文件拷贝到工程目录下，如图 0-25 所示，将对应文件加入工程。

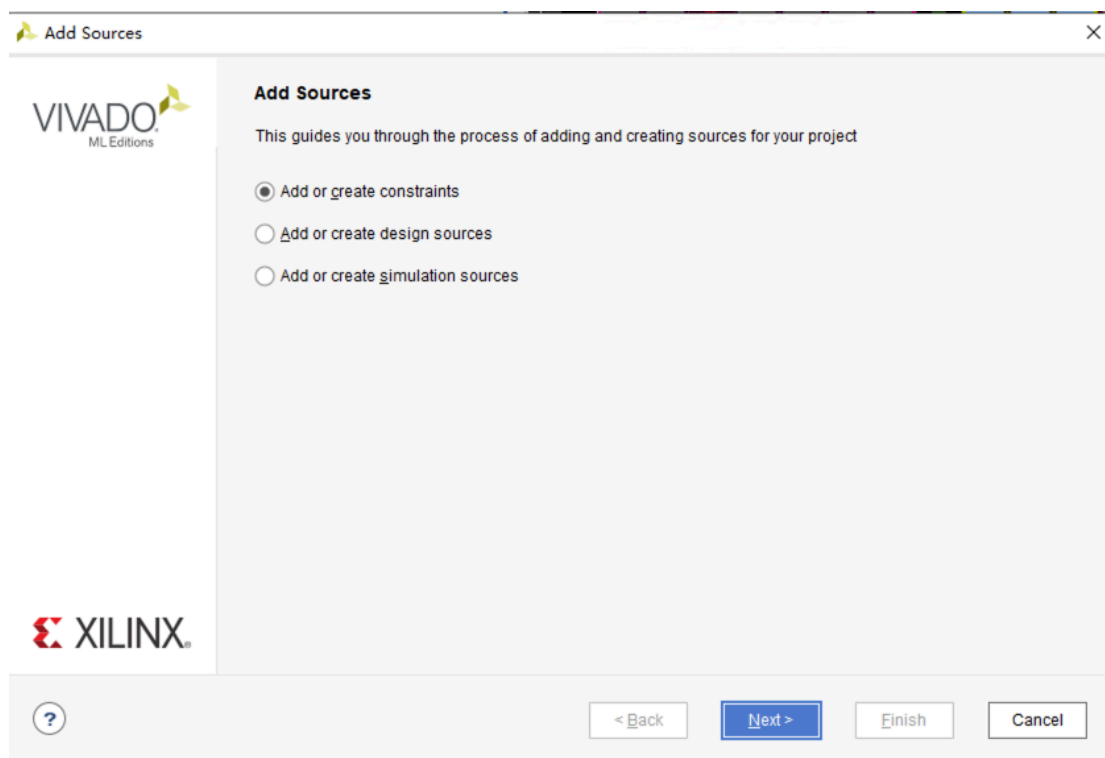


图 0-24: 选择添加约束文件

缺省约束文件中包含了所有 Nexys A7 100T 的引脚定义，但是均是以 # 注释掉的，所以我们需要点击 Source 中 Constraints 下 constrs_1 里的 nexysa7.xdc 进行修改，如图 0-26 所示。

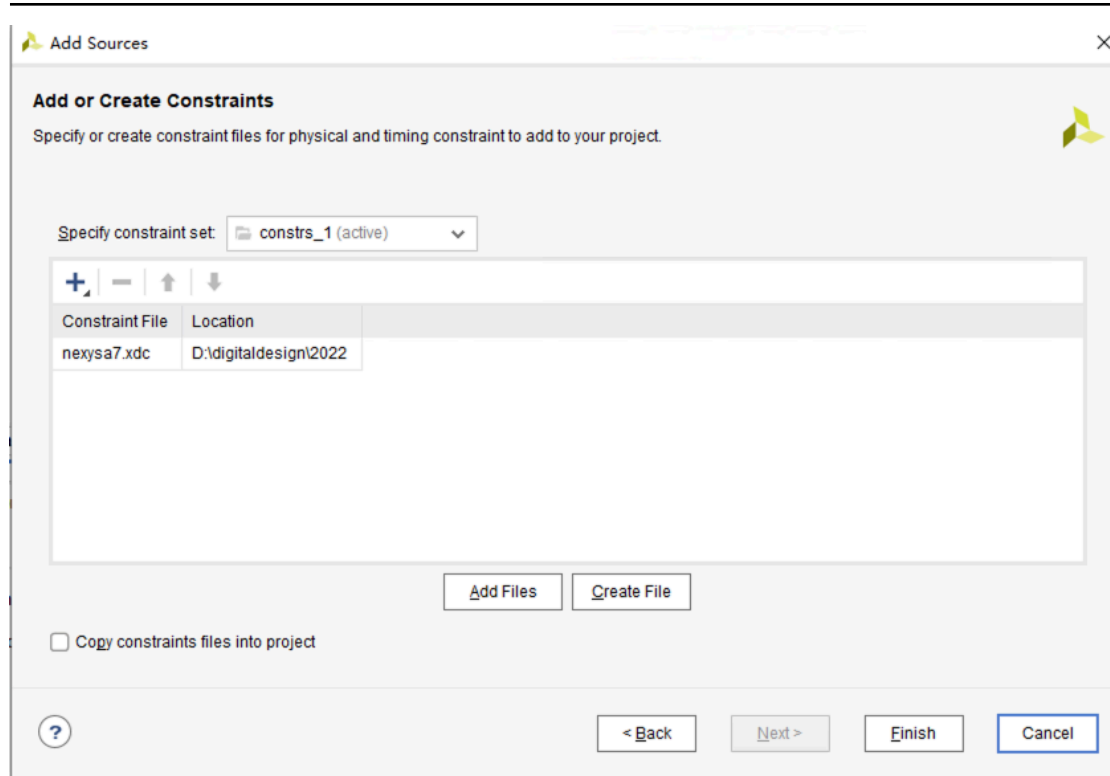


图 0-25: 导入约束文件

约束文件的每一行对应了 **FPGA** 的给定引脚，并给出该引脚与设计中顶层文件的输入输出的对应关系，以及引脚的信号电平的定义。

例如：

```
set_property -dict { PACKAGE_PIN J15    IOSTANDARD LVCMOS33 } [get_ports {SW[0]}];
```

上述语句对顶层文件的 **SW[0]** 信号进行了如下定义：首先，将该信号对应到 **FPGA** 的物理引脚 **J15** 上 (**PACKAGE_PIN J15**)，再通过 (**IOSTANDARD LVCMOS33**) 定义该引脚的电压为 3.3V。

在本实验中，我们应当将输入 **A** 和 **B** 分别连接到拨动开关 **SW0** 和 **SW1** 上，输出 **F** 连接到 **LED0** 上，所以我们可以将以下三行定义开 **Switch** 和 **LED** 的内容进行对应修改。

```
##Switches
set_property -dict { PACKAGE_PIN J15    IOSTANDARD LVCMOS33 } [get_ports { A }];
    #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16    IOSTANDARD LVCMOS33 } [get_ports { B }];
    #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]

## LEDs
```

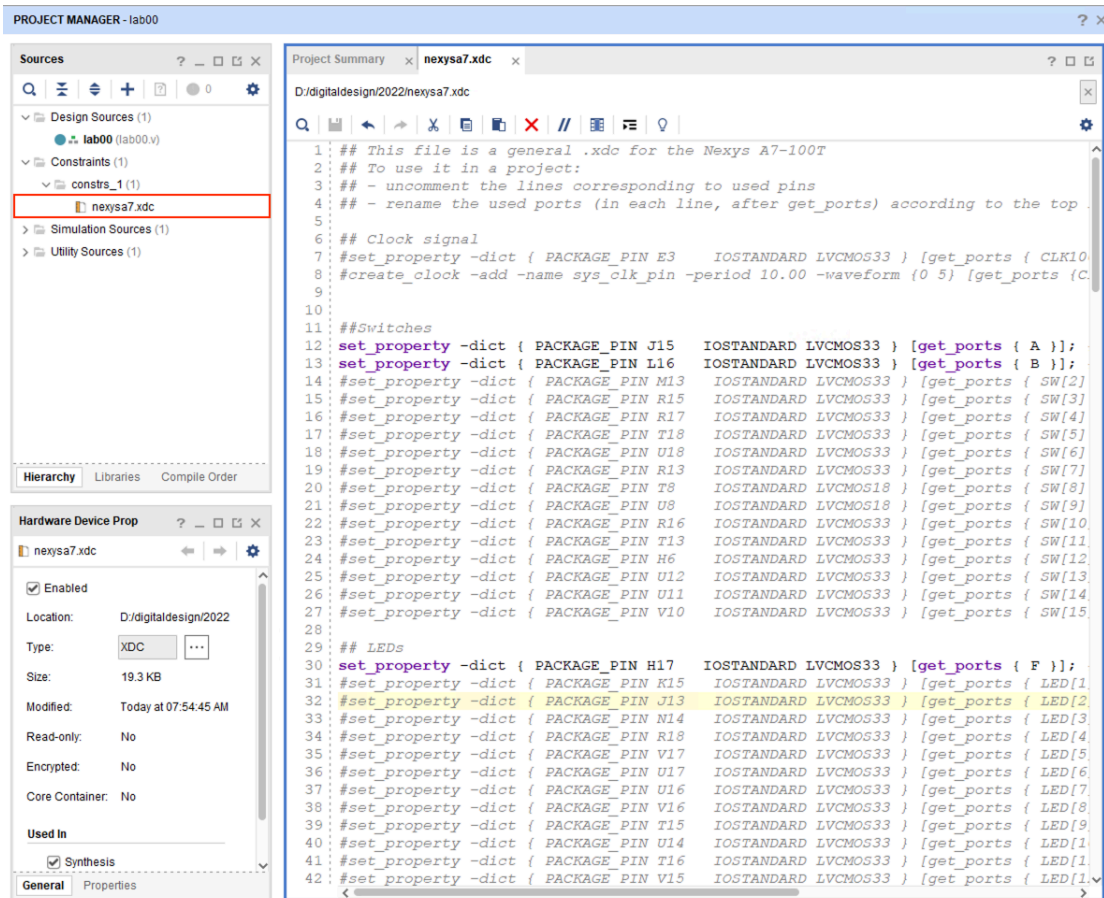


图 0-26: 修改约束文件

```
set_property -dict { PACKAGE_PIN H17    IOSTANDARD LVCMOS33 } [get_ports { F }];
#IO_L18P_T2_A24_15 Sch=led[0]
```

在后续的实验中，如果有大量的开关或 LED 用于输入输出，可以批量地去掉对应输入输出的注释符。这样，缺省定义的 SW 为总线形式，可以在顶层文件的接口中直接用：

```
input [15:0] SW;
```

定义所有的拨动开关，在代码中 SW[i] 即对应第 i 个拨动开关。

方法二: 利用 IO Planning 在流程导航窗口中，单击 Run Synthesis 菜单，完成对工程的综合后，单击 Open Synthesized Design，打开综合设计页面。如果没有出现引脚分配视图，请在菜单 **Layout** 里选择 **I/O Planning**，出现以下 Ping Planing 界面：

在右下方结果窗口中的 I/O Ports 栏，可以看到有 2 个输入 IN 端口和 1 个

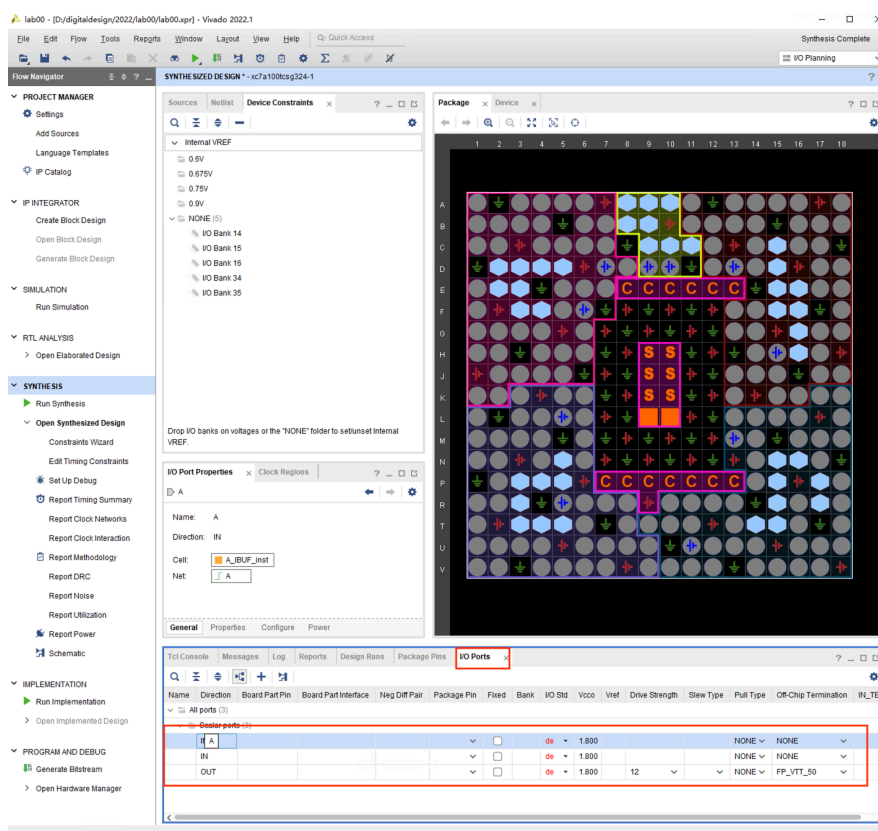


图 0-27: 引脚分配界面

输出 OUT 端口，当鼠标悬停在 IO 端口行上面时，显示该行对应的输入输出变量名称。在对应的输入输出信号后输入对应的 FPGA 引脚标号，并设定 I/O Std 为:LVCMOS33。引脚标号可以查询 Nexys A7-100T 开发板手册。例如，SW0 和 SW1 对应的引脚分别是 J15 和 L16, LED0 的引脚是 H17, 如图 0-28 所示。输入输出信号引脚约束配置结束后，点击界面左上方的保存按钮，则弹出输入新建 XDC 文件名或选择已有的 XDC 文件的对话框。选择 Create a new file，输入文件名为:lab00，单击 **OK** 完成约束文件生成。在 Sources 窗口中的 Constraints 中，可以看到新建的 XDC 文件及其内容。其设置应与之前缺省文件的设置类似。

0.8.2 完成实现

完成工程实现后，将弹出对话框，同样有三个选择：

1. Open Implemented Design 打开实现后的设计

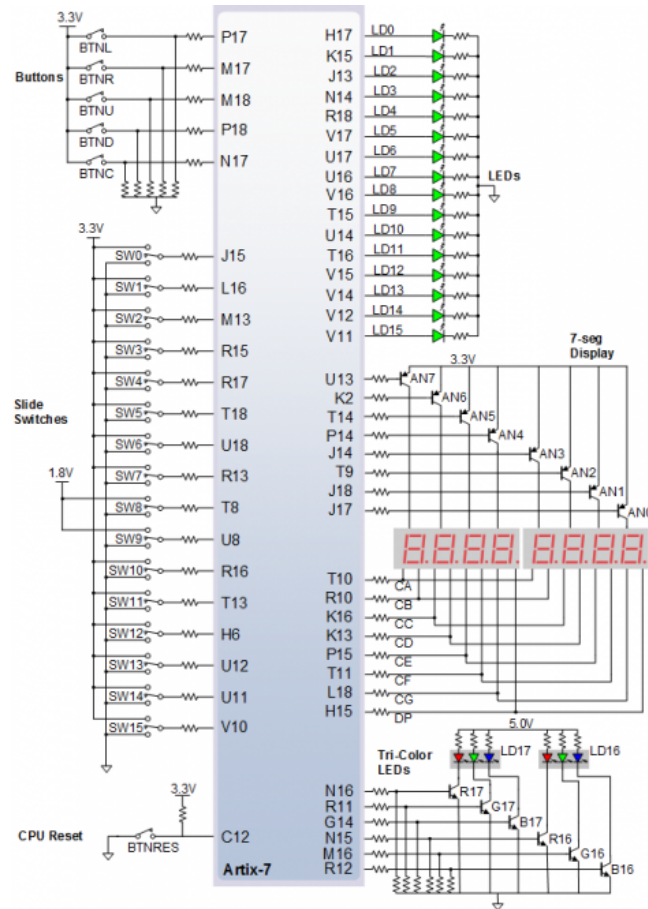


图 0-28: Nexys A7-100T 引脚编号

2. Generate Bitstream 生成烧写 FPGA 的二进制流文件

3. View Reports 查看报告

可以类似综合完成后打开实现后的设计，观察相关报告。也可以直接进行下一步生成二进制流。

0.9 硬件验证

0.9.1 生成二进制文件

实现过程没有出现问题即生成 FPGA 二进制流文件。在工程实现完成的对话框中选择 **Generate Bitstream** 选项或者在流程导航窗口中点击 **Program and Debug** 下的执行 **Generate Bitstream** 命令，工程会自动完成综合、实现、Bit 文

件生成过程 (过程缓慢，时间比较长)，完成之后弹出对话框有四个选择

1. **Open Implemented Design** 打开实现后的设计
2. **View Reports** 查看报告
3. **Open Hardware Manager** 打开硬件管理器
4. **Generate Memory Configuration File** 生成内存配置文件

我们可以先查看报告，判断二进制文件生成是否有问题。

0.9.2 连接开发板

编译完成之后就可以把电路下载到 FPGA 开发平台上验证其正确性了。在编程 FPGA 开发板前首先需要：

- 把 Nexys A7 100T 开发板接上电源，并通过 USB 线接到电脑上。
- 确认已经通过 JP3 跳线正确的配置了电源输入方式为 USB（本学期实验可以不用外接电源），以及通过 JP1 跳线正确的配置了 JTAG 下载方式，如图 0-29 所示。
- 打开开发板电源开关，开发板电源灯会亮。正常情况下开发板预先烧写的代码会启动，LED 及七段数码管会闪动。

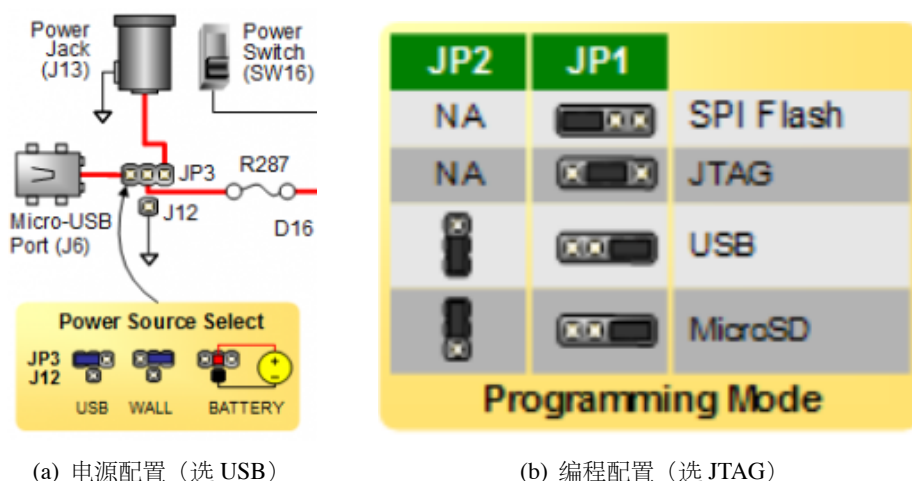


图 0-29: 跳线设置

0.9.3 写入硬件

在开发板接通电源，主机上安装好驱动程序之后。在流程导航窗口中执行 Open Hardware Manager 命令或者在 bit 文件生成完成界面中选择 Open Hardware Manager 选项，点击 OK 按钮，进入硬件编程管理界面。单击 Open Target，选择 Auto Connect，来建立和实验开发板之间的连接。Vivado 将自动开始检测硬件设备，检测到 JTAG 扫描链上的 Artix-7 100T FGPA，如图 0-30 所示。

在目标的 FPGA 器件上用鼠标右击，选择 Program Device... 命令或者在流程导航窗口中的 Open Hardware Manager 中点击 Program Device 命令，将弹出芯片编程文件确认对话框，指定所需的 bit 文件路径，如图 0-31 所示。

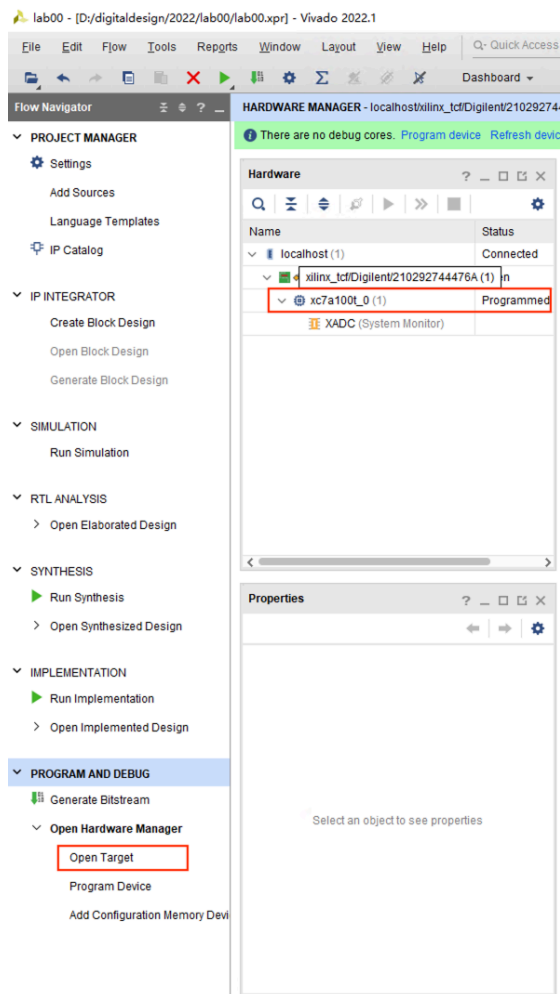


图 0-30: 写入硬件界面

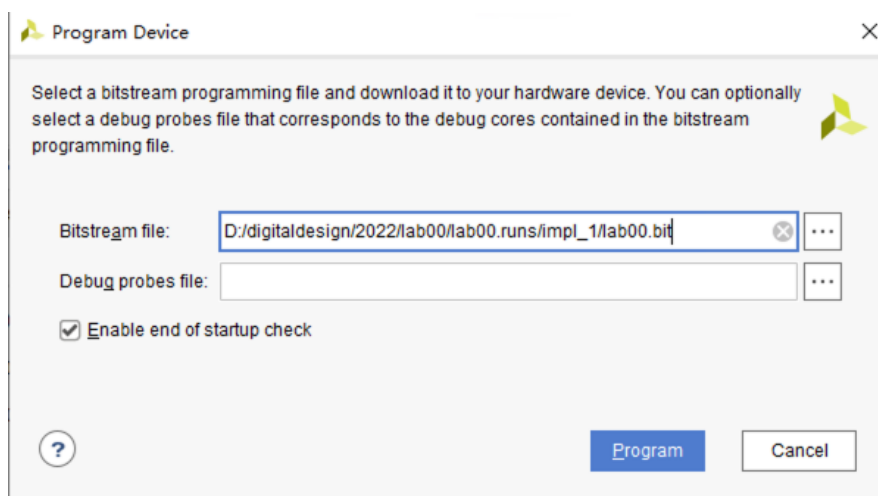


图 0-31: 选择写入文件

点击 **Program**，将 bit 文件下载到开发板卡上的 FPGA 中，下载结束后，在 FPGA 芯片名称后面显示 Programmed。

 在选择 bit 文件时请注意确认，如果硬盘上有多个不同的 bit 文件，硬件中执行的内容由写入的 bit 文件决定，与当前打开的工程无关。

0.9.4 硬件验证

等待 Hardware Manager 将 bit 文件烧写进 XilinxA7-100T 后，将首先会看到开发板上表示烧写完成的 Done 指示会点亮 (LD21)，接着拨动开关 SW0 和 SW1，观察开发板的指示灯 LD0 亮灭情况，验证电路是否满足设计要求。

0.10 实验验收内容

0.10.1 上板实验

本实验无需线下验收，请同学们自行验证工程实现的正确性。

0.10.2 在线测试

请同学们登录南京大学计算机科学与技术系在线实验平台<http://114.212.10.241/>并使用以下邀请码加入数字逻辑与计算机组成原理实验 2026 年秋季班。

● QSJCY

本次实验请完成在线测试的 A XOR B 问题，熟悉在线测试系统，本实验在线测试不计分。